Daniil Barilotti.

Projektportfolio

IT- und Softwareentwicklung mit einem Fundament in Cybersicherheit. Sieben ausgewählte Projekte: responsive Websites, React-Anwendungen, Filmentdeckung und eine Sicherheitsoberfläche.

Alfeld (Leine), Deutschland
React · TypeScript · JavaScript · SCSS · REST · Git

Jedes Kapitel zeigt Oberfläche, Umfang, Architektur und zentrale Entscheidungen. Das Hochschulpraktikum bei DevBrother ist im PromptGuard-Kapitel dokumentiert.

Navigation: Ein Kapitel in der Übersicht auswählen oder die PDF-Lesezeichen öffnen. Die Projekt-Lesezeichen sind eingeklappt. Die separate HTML-Version bietet ein- und ausklappbare Projektblöcke.

Stand: 2. November 2026
Kontakt: daniilbarilotti@gmail.com

Selected work.

Sieben Projekte / klickbare Kapitel

01 Phone Catalog

React · TypeScript · Redux

Frontend-Lernprojekt

02 CineDrop

React · TypeScript · Workers

Eigenes Webprojekt · öffentliche Demo

03 PromptGuard UI

React · Vite · API

Hochschulpraktikum · Frontend-Beitrag

04 FORNO Pizzeria

HTML · CSS · JavaScript

Eigenes Restaurantkonzept · Portfolioentwurf

05 To-do App

React · TypeScript · REST

Frontend-Lernprojekt

06 2048

JavaScript · SCSS

JavaScript-Lernprojekt

07 Bose Landing Page

HTML · SCSS · BEM

Layoutübung nach Designvorgabe

CineDrop: öffentliche Demo, privater Anwendungsquellcode. Projektbeschreibungen und Screenshots sind hier vollständig lesbar.

Phone Catalog

React · TypeScript · Redux

Eine zusammenhängende Shop-Oberfläche statt einzelner UI-Übungen: vom Einstieg über Produktvarianten bis zum Warenkorb. Das Projekt zeigt Komponenten, Routing und persistente Zustandsverwaltung.

FUNKTIONSUMFANG

- Kategorien für Smartphones, Tablets und Zubehör.
- Detailseiten mit Farbe, Speicher und Produktspezifikationen.
- Sortierung und Pagination über URL-Parameter.
- Warenkorb-Mengen und Favoriten über localStorage.

Projektart: Frontend-Lernprojekt

Phone Catalog

Technischer Einblick

ARCHITEKTUR

React-Komponenten und Seiten trennen Darstellung und Navigation. Redux Toolkit verwaltet Produkte, Warenkorb und Favoriten. Services laden den Katalog aus statischen JSON-Dateien.

ENTSCHEIDUNGEN

URL-Parameter halten Listenansichten teilbar. Der Warenkorb speichert Produkt-ID und Menge, statt vollständige Produktobjekte zu duplizieren. Wiederverwendbare Produktkarten vereinheitlichen Listen und Empfehlungen.

DEMO-ABLAUF

Kategorie öffnen, Sortierung ändern, ein Produkt auswählen, Variante wechseln und zum Warenkorb hinzufügen. Nach einem Reload die gespeicherte Auswahl kontrollieren.

PROJEKTGRENZEN

Frontend-Demo ohne Bezahlung, Bestellabwicklung oder Benutzerkonten. Persistenz gilt pro Browser. Der Hero nutzt nun eine statische responsive Produktkomposition statt des alten Sliders. Produktionsbuild, TypeScript und Komponenten-Lint bestehen; die veröffentlichte Desktopansicht wurde geprüft.

QUELLCODE & DEMO

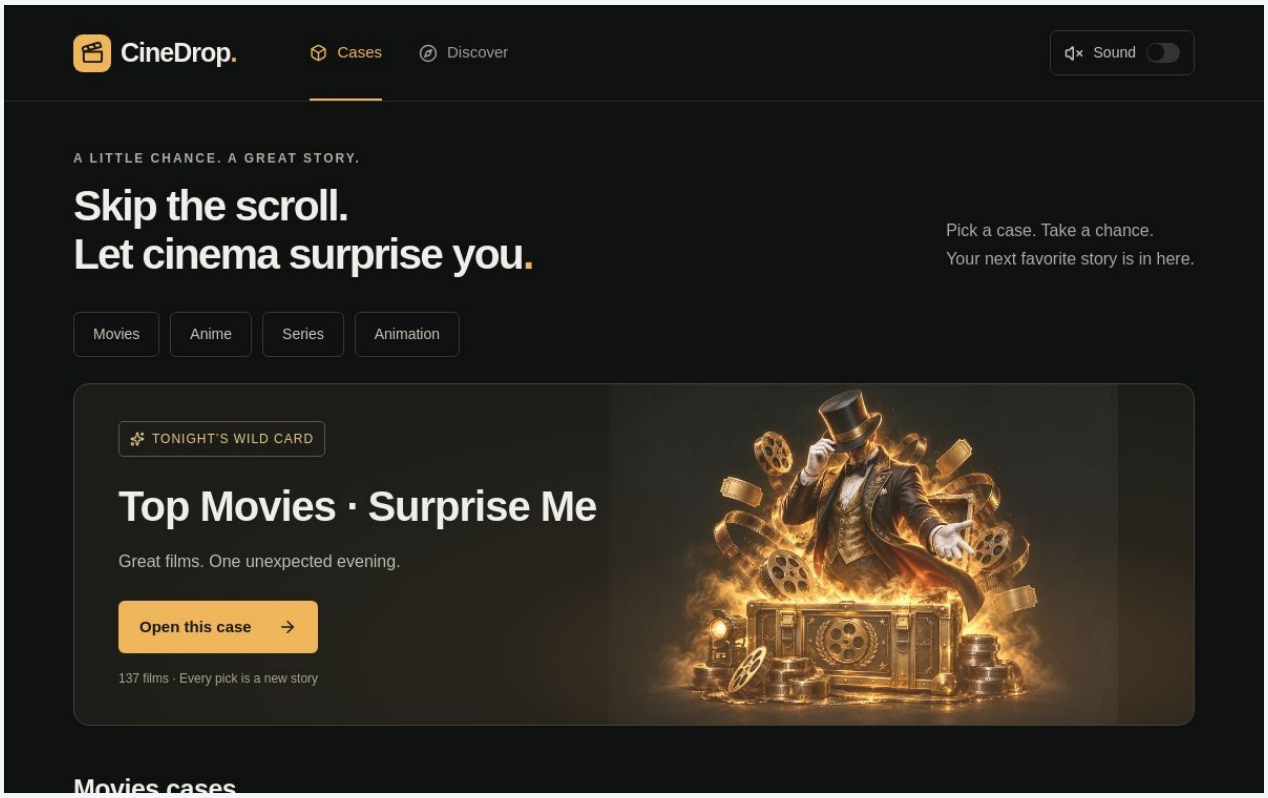
https://github.com/DaniilBarilotti/phone-catalog

https://daniilbarilotti.github.io/phone-catalog/

CineDrop

React · TypeScript · Workers

CineDrop hilft bei der Filmauswahl: 24 Themenboxen kombinieren eine animierte Zufallsauswahl mit einem durchsuchbaren Katalog von 12.759 Filmen. Serien und Anime haben eigene Ansichten.



Gespeicherte Ansicht der veröffentlichten Version vor der Wartung · September 2026

FUNKTIONSUMFANG

- Themenboxen für Filme, Serien, Anime und Animation.
- Suche nach Titel, Darsteller oder Regisseur, Filter und Sortierung.
- Zufallsauswahl mit Schnellmodus, Tonoption und reduzierter Bewegung.
- Detailseiten mit Quellenangaben und Rückkehr zur Suche.

Projektart: Eigenes Webprojekt · öffentliche Demo

CineDrop

Technischer Einblick

ARCHITEKTUR

React und TypeScript bilden die Oberfläche. Vinext läuft auf Cloudflare Workers. Lokale JSON-Kataloge versorgen Filme und Themenboxen; API-Routen normalisieren TVmaze- und AniList-Daten, cachen Antworten und behandeln Ausfälle.

ENTSCHEIDUNGEN

Die Auswahl erfolgt unabhängig von der Verfügbarkeit externer Poster. Ersatzkarten halten den Ablauf nutzbar. Bei mehreren Titeln wird eine direkte Wiederholung vermieden. Suchparameter bleiben in der URL und im Rücksprung aus der Detailseite erhalten.

DEMO-ABLAUF

Eine Themenbox öffnen, Schnellmodus oder Animation wählen und einen Titel aufrufen. In Discover suchen, Genre ändern, eine Detailseite öffnen und zur gleichen Suche zurückkehren.

PROJEKTGRENZEN

Die Demo ist öffentlich; der Anwendungsquellcode bleibt privat. Externe Daten- und Bilddienste können ausfallen; Bewertungen sind mit Quelle und Datum gekennzeichnet. Kein Filmstreaming. TypeScript, Regressionstests und Build bestehen; die Änderungen wurden nicht im Browser auf einem Mobilgerät geprüft.

QUELLCODE & DEMO

https://cinedrop-beta.vercel.app

PromptGuard UI

React · Vite · API

Hochschulpraktikum bei DevBrother (29. Juni–19. Juli 2026). Der eigene Beitrag zur Teamlösung PromptGuard umfasst die React-Oberfläche: Chat, Guardrail-Status, verdächtige Textstellen und eine Vorfallübersicht.

FUNKTIONSUMFANG

- Markdown-Darstellung für Chat-Antworten.
- Zustände für saubere, verdächtige und blockierte Anfragen.
- Markierte Textfragmente und Angriffsmetadaten.
- Vorfallübersicht, Beispielprompts, Sprach- und Themewechsel.

Projektart: Hochschulpraktikum · Frontend-Beitrag

PromptGuard UI

Technischer Einblick

ARCHITEKTUR

Der API-Client übersetzt Proxy-Antworten in UI-Zustände. Ein Chat-Hook verwaltet Nachrichten und Anfragen. Separate Komponenten bilden Chat, Sicherheitsübersicht und Red-Team-Bereich. Der Node-Proxy und Python-Detektor liegen im Team-Repository.

ENTSCHEIDUNGEN

Eine zentrale Normalisierung hält Backend-Details aus Darstellungs-Komponenten heraus. HTTP 403 wird als fachliches Guardrail-Ergebnis angezeigt; Rate-Limits werden separat als Fehler behandelt.

DEMO-ABLAUF

Chat-Ansicht und Red-Team-Beispiele öffnen. Den Weg eines Proxy-Ergebnisses von src/api/client.js bis zur Statusanzeige nachvollziehen.

PROJEKTGRENZEN

Die eigene Arbeit in diesem Repository ist das Frontend. Der standalone Mock-Modus wechselt vordefinierte Ergebnisse unabhängig vom eingegebenen Text. Er ist kein Detektor und keine Sicherheitsgarantie.

QUELLCODE & DEMO

https://github.com/DaniilBarilotti/promptguard-ui

https://promptguard-ui.vercel.app

FORNO Pizzeria

HTML · CSS · JavaScript

Ein bewusst reduzierter Restaurantauftritt mit eigener visueller Richtung: warme Papierfarbe, Tomatenrot, große Typografie und klare Menüführung.

FUNKTIONSUMFANG

- Vollständige deutsche und englische Textansichten.
- Responsives Hero-Layout und lesbare Speisekarte.
- Native Zutaten-Akkordeons mit Tastaturbedienung.
- Sichtbare Fokuszustände und reduzierte Bewegung.

Projektart: Eigenes Restaurantkonzept · Portfolioentwurf

FORNO Pizzeria

Technischer Einblick

ARCHITEKTUR

Statische HTML-Seite, CSS-Design-Tokens und ein kleiner JavaScript-Sprachwechsler. Originale Textknoten bleiben erhalten, damit wiederholtes Umschalten keine Inhalte verliert.

ENTSCHEIDUNGEN

Native HTML-Elemente übernehmen Navigation und Akkordeons. Ohne Framework und Build-Schritt bleibt die Seite leicht lokal zu öffnen. Die Sprache wird nach Möglichkeit auf dem Gerät gespeichert.

DEMO-ABLAUF

Sprache wechseln, zur Speisekarte navigieren und Zutaten aufklappen. Fokus mit der Tab-Taste kontrollieren.

PROJEKTGRENZEN

Das Restaurant ist fiktiv. Speisekarte und Preise sind Beispiele. Es gibt keinen Buchungs- oder Bestellservice und keine behaupteten Kundenresultate. Fotoquellen und Lizenzen bleiben im README dokumentiert.

QUELLCODE & DEMO

https://github.com/DaniilBarilotti/forno-pizzeria

https://daniilbarilotti.github.io/forno-pizzeria/

To-do App

React · TypeScript · REST

Eine kleine Anwendung mit vollständigen asynchronen Aufgabenabläufen. Der Schwerpunkt liegt auf konsistentem UI-Zustand bei Laden, Bearbeiten und Fehlern.

FUNKTIONSUMFANG

- Aufgaben erstellen, bearbeiten und löschen.
- Erledigungsstatus ändern.
- Ansichten für alle, offene und erledigte Aufgaben.
- Lade- und Fehlerrückmeldungen bei API-Aufrufen.

Projektart: Frontend-Lernprojekt

To-do App

Technischer Einblick

ARCHITEKTUR

Ein Fetch-Client sendet GET, POST, PATCH und DELETE an die externe Mate Academy API. Typisierte Komponenten bilden die Aufgabenansicht. useTodo kapselt den Bearbeitungszustand und die Eingabereferenz.

ENTSCHEIDUNGEN

Titel werden vor dem Speichern getrimmt. Ein leerer bearbeiteter Titel folgt dem Löschablauf. Nach fehlgeschlagenem Speichern erhält die Eingabe erneut den Fokus.

DEMO-ABLAUF

Eine Aufgabe erstellen, umbenennen, erledigen und filtern. Bei einem fehlgeschlagenen Request prüfen, ob Eingabe und Fehlerfeedback verständlich bleiben.

PROJEKTGRENZEN

Aufgaben liegen in einer externen Lern-API, nicht ausschließlich in localStorage. Diese Anwendung enthält keinen eigenen Backend-Service und keine eigenständige Benutzerverwaltung. API-Verfügbarkeit beeinflusst die Demo.

QUELLCODE & DEMO

https://github.com/DaniilBarilotti/todo-app

https://daniilbarilotti.github.io/todo-app/

2048

JavaScript · SCSS

Eine Implementierung des bekannten 2048-Prinzips, die Array-Transformationen und einen klaren Spielzustand sichtbar macht. Die eigene Leistung ist die Umsetzung, nicht die Erfindung des Spielkonzepts.

FUNKTIONSUMFANG

- 4 × 4 Spielfeld mit zufälligen neuen Kacheln.
- Bewegung und Zusammenführung gleicher Werte.
- Punktestand und Gewinn-/Verlustzustände.
- Start, Neustart und Touch-Steuerung per Wischgeste.

Projektart: JavaScript-Lernprojekt

2048

Technischer Einblick

ARCHITEKTUR

Das Spielfeld ist ein zweidimensionales Array. Die Zeilenfunktion entfernt Nullen, verbindet gleiche Nachbarn einmal und füllt leere Stellen wieder auf. Vertikale Züge wenden dieselbe Logik auf Spalten an.

ENTSCHEIDUNGEN

Ein Vergleich vor und nach dem Zug verhindert neue Kacheln bei wirkungslosen Bewegungen. Richtungslogik wird über Umkehrung von Zeilen oder Spalten wiederverwendet.

DEMO-ABLAUF

Start wählen und mit Pfeiltasten oder Wischgesten spielen. An der Zeile [2, 2, 2, 2] erklären, warum der nächste Zustand [4, 4, 0, 0] ist.

PROJEKTGRENZEN

Tastatur- und Touch-Steuerung; noch keine gespeicherten Partien. Wischrichtung, Mindestdistanz und Abbruch werden durch Regressionstests geprüft; kein Test auf einem physischen Smartphone. Starter-Credits und GPL-3.0-Lizenz bleiben erhalten. Die aktuelle Korrektur ignoriert Pfeiltasten vor Spielstart und fremde Tasten.

QUELLCODE & DEMO

https://github.com/DaniilBarilotti/2048_game

https://daniilbarilotti.github.io/2048_game/

Bose Landing Page

HTML · SCSS · BEM

Eine Produkt-Landingpage, die eine vorgegebene Gestaltung in semantisches HTML und responsive SCSS-Komponenten überträgt.

FUNKTIONSUMFANG

- Hero, Produktempfehlungen und Produktkategorien.
- Mobile Navigation und Ankerlinks.
- Kontaktformular-Layout mit nativer Validierung.
- BEM-Klassen und wiederverwendbare SCSS-Hilfen.

Projektart: Layoutübung nach Designvorgabe

Bose Landing Page

Technischer Einblick

ARCHITEKTUR

Statische HTML-Struktur mit getrennten SCSS-Dateien für Header, Menü, Galerie, Produkte und Formular. Mixins und Variablen vereinheitlichen Raster und responsive Abstände.

ENTSCHEIDUNGEN

Die Gestaltung wird in kleine benannte Bereiche zerlegt. Native Links und Formulare behalten erwartbares Browser-Verhalten. Das Projekt demonstriert Layout-Handwerk neben den React-Projekten.

DEMO-ABLAUF

Menü auf schmaler Ansicht öffnen, zu Produktbereichen navigieren und das Formular-Layout prüfen.

PROJEKTGRENZEN

Übungsprojekt ohne Verbindung zur Marke Bose. Das Kontaktformular hat keinen Versand-Backend. Der Repositoryname miami-landing stammt aus dem ursprünglichen Starter und beschreibt den tatsächlichen Bose-Inhalt nicht.

QUELLCODE & DEMO

https://github.com/DaniilBarilotti/miami-landing

https://daniilbarilotti.github.io/miami-landing/